11 BUILT-IN PIPES vs CUSTOM PIPES — SIDE BY SIDE

Both are the exact same mechanism — a class implementing PipeTransform, running before your handler (see theory/09-pipes.txt). The only difference is WHO WROTE THE RULE and HOW GENERAL it is.

```
ParseIntPipe        -> written by Nest, ships in @nestjs/common,
                       covers a GENERIC rule ("string -> int")
                       that's the same in every app that uses it.

TaskExistsPipe       -> written by YOU, lives in your project,
(see theory/10-custom-pipes.txt)  covers a rule SPECIFIC to this
                       app's domain ("this task id must exist").
```

REAL EXAMPLE FROM THIS PROJECT

```
// task.controller.ts — BUILT-IN pipe in use today
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.taskService.findOne(id);
}

// theory/10-custom-pipes.txt — CUSTOM pipe, same route, designed
// (not yet added to the actual controller — see note at bottom)
@Get(':id')
findOne(@Param('id', TaskExistsPipe) task: Task) {
  return task;
}
```

Both run at the exact same moment in the request lifecycle (Pipes step, right before the handler — see theory/09-pipes.txt's ordering diagram). The difference is what each one is ALLOWED to know and do.

SIDE-BY-SIDE TABLE

```
|                    | Built-in (ParseIntPipe)      | Custom (TaskExistsPipe)         |
|--------------------|------------------------------|---------------------------------|
| Who wrote it        | Nest team, in @nestjs/common  | You, in your own project         |
| Rule scope          | Generic — same for every app | Specific to THIS app's domain    |
| Knows about your DB/ | No — pure string->type logic | Yes — can inject a service and   |
| services?           |                               | look something up                |
| Can use DI          | N/A (stateless, no deps)     | Yes — it's @Injectable(), so it  |
|                     |                               | can constructor-inject TaskService|
| Output              | A converted primitive         | Can be an entire domain object   |
|                     | (number, boolean, UUID string)| (a full Task, not just its id)   |
| Where it lives       | node_modules (installed)      | src/task/pipes/ (your codebase)  |
| Reusable across apps? | Yes, drop into any project    | No — tied to this app's TaskService|
```

WHAT BUILT-IN PIPES CAN'T DO (WHY CUSTOM ONES EXIST)

ParseIntPipe genuinely has no idea what a "Task" is — it only knows "convert this string to a number, or throw." It CANNOT:

- check whether that number corresponds to a real task
- fetch anything from a service/database
- enforce a domain rule like "must be one of these three statuses"

That's the entire reason theory/10-custom-pipes.txt's TaskExistsPipe exists — it wraps ParseIntPipe's job (string → number) AND adds a domain-specific step (number → real Task, or throw 404) that no built-in pipe could ever know how to do, because it requires knowledge of THIS app (TaskService) that Nest's shipped pipes don't have.

WHAT BUILT-IN PIPES ARE BETTER FOR

For pure format/type conversion with no domain knowledge required (string → int/float/bool/UUID, or checking a value against a real TS `enum`), always prefer the built-in over writing your own — less code, already tested, and instantly recognizable to anyone who knows Nest:

@Param('id', ParseIntPipe)	NOT a hand-rolled string→number pipe
@Param('id', ParseUUIDPipe)	NOT a hand-rolled UUID regex check
@Query('status', new ParseEnumPipe(Status))	NOT a manual includes() check

Reach for a custom pipe only once the rule needs something a built-in structurally cannot express — domain knowledge, a DB/service lookup, or app-specific business logic.

THEY COMPOSE — NOT EITHER/OR

Nothing stops mixing both kinds on the same handler, or even chaining a built-in INSIDE a custom one (as TaskExistsPipe does — it does the same `parseInt` job ParseIntPipe does, then goes further):

```
@Put(':id')
update(
  @Param('id', ParseIntPipe) id: number,     // built-in: string->number
  @Body() dto: UpdateTaskDto,                 // built-in (ValidationPipe,
) { ... }                                     // registered globally)
```

A real app typically has MANY built-in pipes in daily use and only a FEW custom ones, written specifically for the handful of rules that are truly unique to that app's domain.

★ RULE OF THUMB

Default to a built-in pipe. Write a custom one only when you catch yourself repeating the same "if (!found) throw NotFoundException()" (or similar domain check) across multiple handlers — that repetition is the signal to lift it into a custom pipe once, the same way repeated @Body('field') params were the signal to introduce a DTO (theory/13-dto.txt).

NOTE TaskExistsPipe above is a DESIGNED example from theory/10-custom-pipes.txt for teaching purposes — it has not actually been added to src/task/ yet. Ask if you'd like it wired into the real task.controller.ts/task.service.ts to test against.

See also: theory/09-pipes.txt, theory/10-custom-pipes.txt